AI+X summer: Custom Topology and Routing for Matrix Multiplication

Instructed by *Kaisheng Ma*

Student A YaoClass xx 20xxxxxxxx
Student B YaoClass xx 20xxxxxxxx

1 Spot Lights

- Design and implement a novel topology for distributed matrix multiplication.
- Implement multicast to accommodate the distributed matrix multiplication scenario.
- Incorporate several tricks to fully exploit our topology.
- Design and conduct detailed and various experiments to validate our implementation.

2 Architecture

2.1 Overview

Our topology is shown in Figure 1.
As shown in the figure, the blue nodes are the actual computation units, the yellow and green nodes are the routing units, and the red node is the control unit. In the matrix multiplication case, all data are sent from the control node to computation units.
As for the links, we can see that both the green and yellow links form binary trees.

2.2 Motivation

The reasons why we design such a topology are highly related to the nature of distributed matrix multiplication.
In distributed matrix multiplication $A \cdot B$, we usually divide matrix A into several row blocks, and divide matrix B into several column blocks. See Figure 2.
In our implementation, there are 2^k rows and columns of computation units, therefore 2^{2k} computation units in total. We assume that matrix A is evenly divided into 2^k row blocks and matrix B is evenly divided into 2^k column blocks. Then the computation unit located in the i-th row and the j-th column requires the i-th row block from matrix A and the j-th column block from matrix B in order to carry out a small-scale matrix multiplication.
We notice that all computation units in the same row require the same row block from matrix A and all computation units in the same column require the same column block from matrix B. Therefore, we design our topology and implement multicast, in order to let the control unit send fewer packets and to reduce network loads.
See Figure 3 for the distributed matrix multiplication's task allocation example.

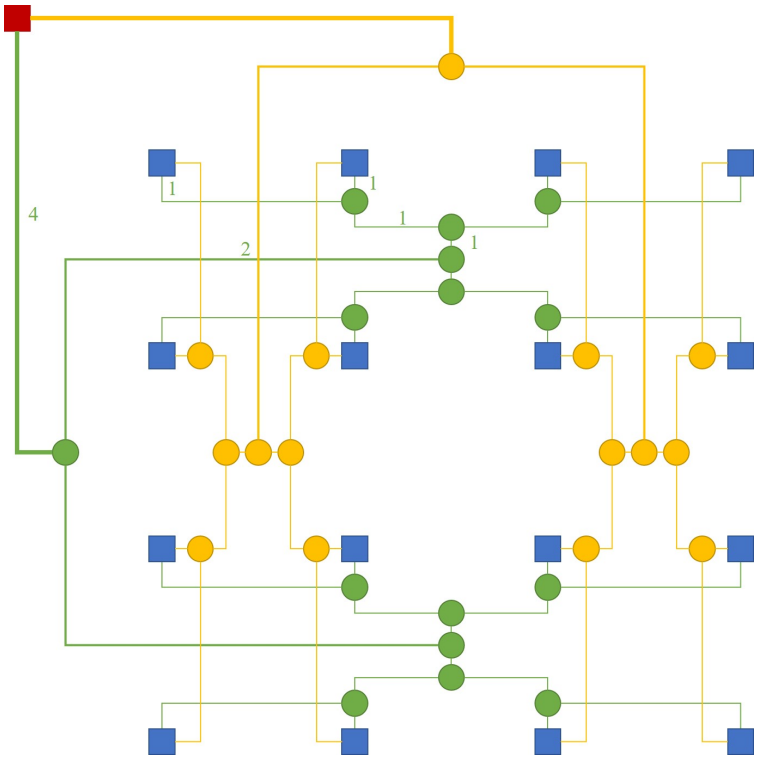


Figure 1: Network Architecture

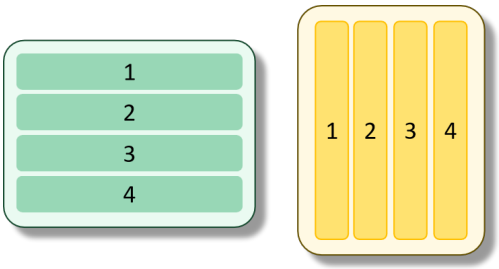


Figure 2: Distributed Matrix Multiplication
Take the $k = 2$ case as an example, we divide the former green matrix into row blocks while the latter yellow matrix into column blocks.

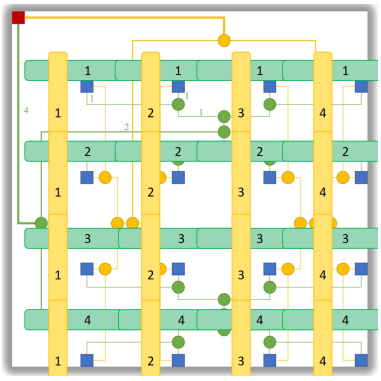


Figure 3: Distributed matrix multiplication's task allocation on our topology

3 Implementation

3.1 Multicast

Because in vanilla distributed matrix multiplication, data are always required to be replicated several times, we implement multicast to enable efficient task allocation.
To implement this, we mainly modify three components, the Routing Unit, the Switch Allocator, and the Crossbar.

3.1.1 Routing Unit

Originally, the output port directions are *East*, *West*, *North*, and *South*, each representing a single direction. To implement multicast, we add *North&South* and *West&East* output ports, which represent combinations of some single ports. In reality, such output ports do not physically exist; data can only be transferred via the single-direction output ports. These combined output ports are used solely for the ease of analysis in the Switch Allocator.
Since matrix multiplication only requires broadcast, we implement this simplified case. However, the combination of output ports can be easily generalized to arbitrary subsets of all output port directions, provided that a mapping from the set to a combined output port is maintained.

3.1.2 Switch Allocator

This is the core part of the multicast implementation. In gem5, one wake-up of the switch allocator includes the arbitration of input ports, the arbitration of output ports, and additional checking and resetting components.
In the part that arbitrates input ports, we maintain an additional counter for the remaining output ports.
In the part that arbitrates output ports, if a flit with a sending request still has remaining output ports, we clone it instead of popping it.
Regarding the credit, the logic of the output unit's credit remains same, while the logic of the input unit's credit changes. The input unit will not send a credit back until the flit has been sent to all target output ports.
Additionally, we should reset the virtual channel after a flit has been sent to each output port. There is no need to keep sending to the same virtual channel for different output ports.

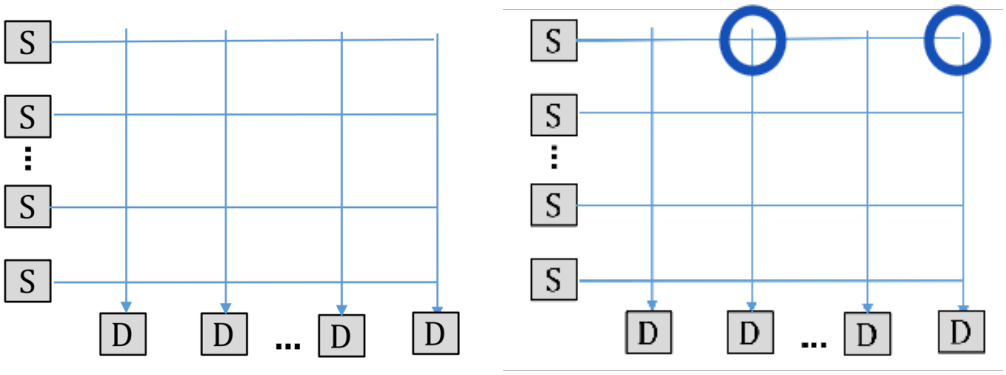
3.1.3 Crossbar

A traditional Crossbar is shown Figure 4. Originally, in one cycle, one source port is connected to only one destination port. In order to improve the performance of multicast, we connect one source port to multiple destination ports in one cycle.
Specifically in the gem5 simulation code, originally each input port has a flit buffer storing the flits to be transferred to output ports. Then in one cycle, an input port can only send a flit to output ports. We now assign these flit buffers to output ports, so that each output port can only receive a flit, but each input port can send a flit to multiple output ports in one cycle.

3.2 Tricks

3.2.1 Vary Link Period

We notice that if all links are of the same capability, then a few links near the control unit will become the bottleneck of the entire topology, as highlighted in Figure 5.



(a) original (b) modified

Figure 4: Modified Crossbar

To improve the performance, we increase the frequency of these bottleneck links, or in other words, decrease their period. Still take the $k = 2$ case for example, we increase the frequency of the top green and yellow edges to 4 times and increase the frequency of the below 2 green and yellow edges to twice.

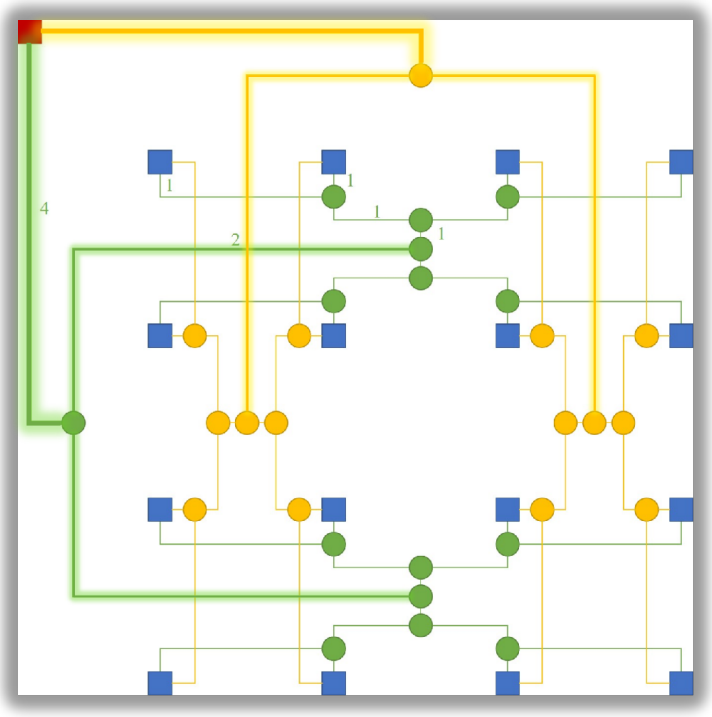


Figure 5: Varied Link Period

In total, we only increase the frequency of $2(2^k - 1)$ links among $2(2^{2k+1} - 1)$ links. The ratio is about $1/2^{k+1}$, which is quite small.

3.2.2 Optimize Traffic

Originally, we send packets from the control unit in a natural order. We first send packets to all rows, and then to all columns. But such a traffic is not balanced. Each link in the tree will be busy and even congested

in some periods, while idle in other periods.
Therefore, we optimize the packet sending order. Take the $k = 2$ case as an example, where there are $2^k = 4$ computation units in a row, instead of the previous traffic $r_0, r_1, r_2, r_3, c_0, c_1, c_2, c_3$, we send packets in the order of $r_0, c_0, r_2, c_2, r_1, c_1, r_3, c_3$, where r stands for row, and c stands for column.
This optimized traffic can balance loads for all links in the binary tree.

4 Evaluation

4.1 Evaluation Methodology

We compare our implementation with MeshXY.

4.1.1 Single Sender

Due to the nature of distributed matrix multiplication, only the control unit is the packet sender. So there is only a single sender in our synthetic traffic.
For Mesh, we let the top left node be the control unit and send packages to all nodes including itself. The packages are sent to all nodes first in the order of all rows, and then in the order of all columns.
For our topology, the control unit is the single sender and the computation units are the destinations. If we enable multicast, the traffic is described in Section 3.2.2.
To be explicit, the method is shown in Figure 6.

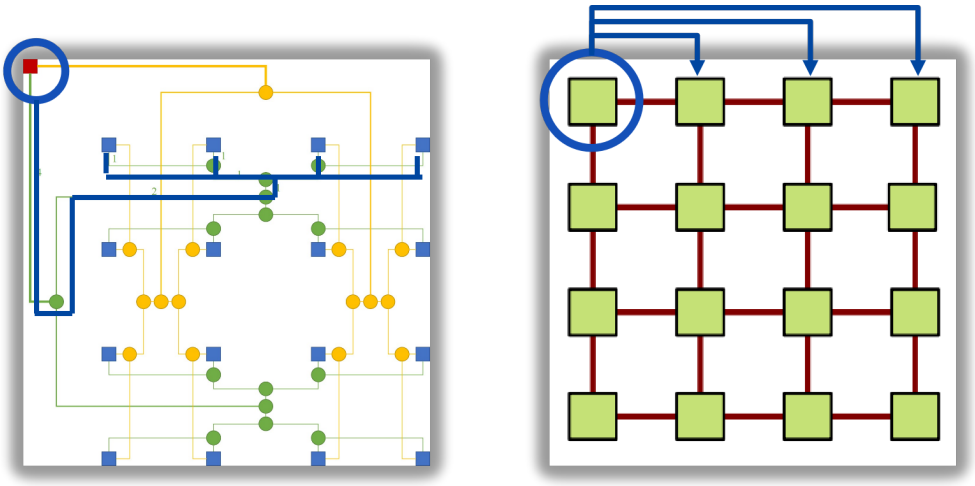


Figure 6: Single Sender

4.1.2 Link Period

In the matrix multiplication case, only the control unit sends packets. Since in each cycle, there is only one packet injected into the network, there will be no congestion. Therefore, to show the performance of our implementation, we add a new variable, *link period*. That is, each link can send a one flit every *link period* cycles instead of every cycle.

4.1.3 Modified Definition of "Injection"

Because of the implementation of multicast, compared with Mesh, our topology need to send fewer packets for completely distributing two matrices for matrix multiplication. Therefore, we need to modify the definition

of "injection" in order to make comparison fair.
In our evaluation, we denote an "injection" as completely distributing the two matrices for a matrix multiplication. We can take the $k = 2$ case as an example. For our topology with multicast, one "injection" is equivalent to sending 8 packets, where 4 packets are sent to rows, and the other 4 packets are sent to columns. And for 4x4 Mesh, one "injection" is equivalent to sending 32 packets, where 2 packets are sent to each computation unit, indicating a row block and a column block.

4.2 Latency & Reception Ratio

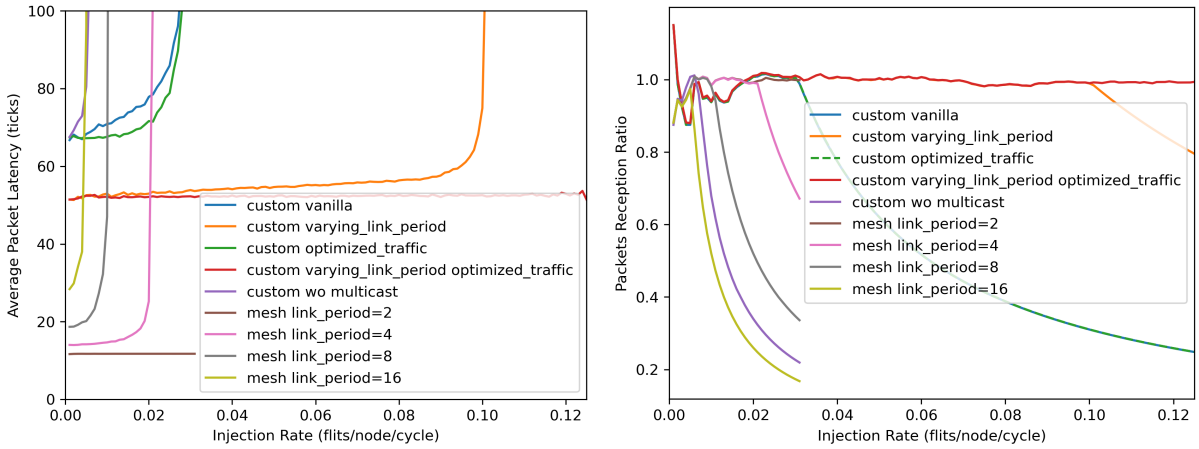
We evaluate average packet latency and reception ratio for different settings of mesh and our topology, as shown in Figure 7 and Figure 8, for $k = 2$ case and $k = 3$ case, respectively.

- **custom vanilla** is the vanilla version of our topology, *i.e.*, with multicast, without optimized traffic and varying link period.
- **custom varying_link_period** increases the frequency of bottleneck links, as explained in 3.2.1.
- **custom optimized_traffic** optimizes the packet sending order, as explained in 3.2.2.
- **custom varying_link_period optimized_traffic** combines the above two tricks, which is the full version of our implementation.
- **custom wo_multicast** disables multicast compared with **custom vanilla**, aiming at validating the effectiveness of multicast.
- **mesh link_period=T** is a mesh where the period of each link is set to T. The smaller T is, the more powerful the network is.

As we can see from the figures, with the same link period, mesh (**mesh link_period=16** for $k = 2$ and **mesh link_period=32** for $k = 3$) quickly gets congested with rapidly increasing latency and decreasing reception ratio. In contrast, the latency and reception ratio of our full version **custom varying_link_period optimized_traffic** remain unchanged as the injection rate increases, which demonstrates the capability of our topology.
Compare **custom vanilla** and **custom varying_link_period** (or **custom optimized_traffic** and **custom varying_link_period optimized_traffic**), we demonstrate that increasing the frequency of only a few bottleneck links can greatly increase the saturation throughput.
Compare **custom vanilla** and **custom optimized_traffic** (or **custom varying_link_period** and **custom varying_link_period optimized_traffic**), we demonstrate that optimizing traffic can also contribute to lower latency and higher saturation throughput by balancing loads.
Compare **custom vanilla** and **custom wo_multicast**, we show that multicast is critical in our design, which can reduce the number of packets to be sent by the control unit, thereby reducing the network loads.
We can also compare **mesh link_period=T** for different T, which shows that only when the link period is small enough can mesh remain uncongested as the injection rate increases.

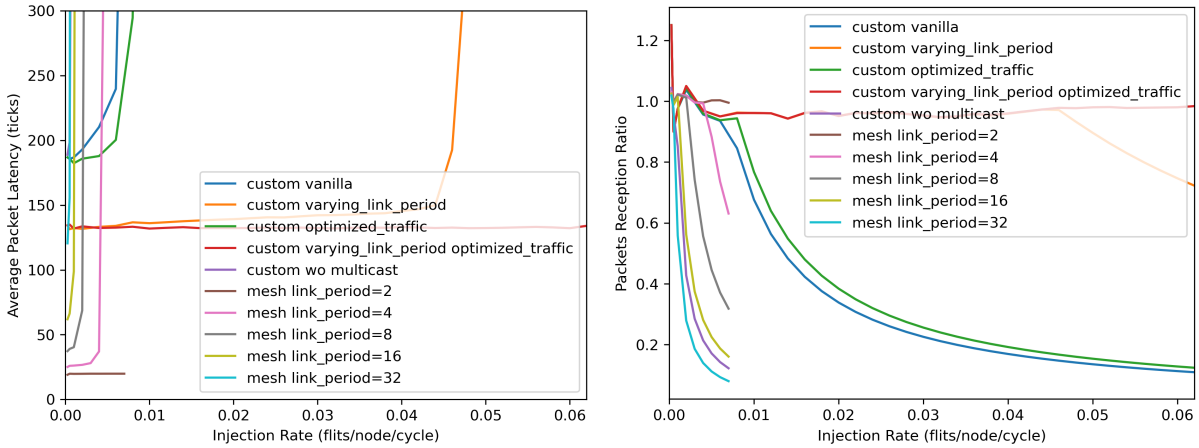
4.3 Hop Count

Figure 9 shows the comparison of average hops between mesh and our topology. Because of the constraints in gem5, we can only test for $k \leqslant 3$ cases.
As we can see, when the network is small ($k = 2$ or $k = 3$), our topology has no advantage over mesh in terms of average hops. However, the average hops of our topology have a much slower growth rate than that of mesh as the network becomes larger. That is because when the network is of size $2^k \times 2^k$, the average hops of our topology are $2k + 1$, while those of mesh are $2^k - 1$. The analytical result is shown in Figure 9(c).



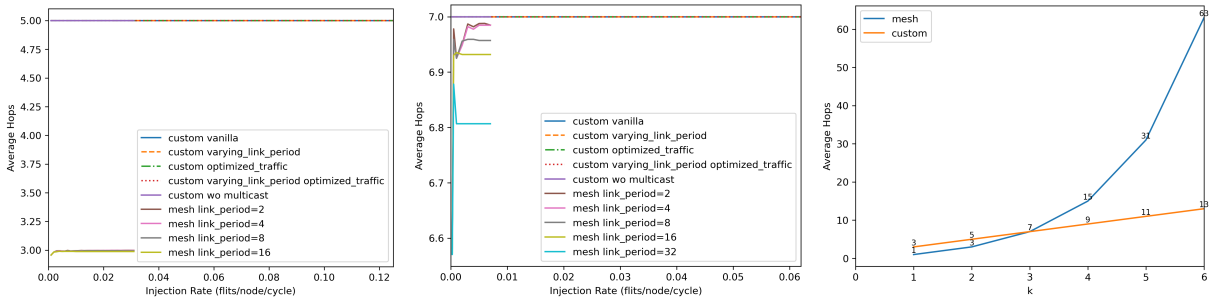
(a) average packet latency (b) reception ratio

Figure 7: Average packet latency and reception rate curves for mesh and our topology for the $k = 2$ case



(a) average packet latency (b) reception ratio

Figure 8: Average packet latency and reception rate curves for mesh and our topology for the $k = 3$ case



(a) $k = 2$ (b) $k = 3$ (c) general k

Figure 9: Experimental results and analytical results of average hops for different network sizes

5 Conclusion

In this project, we design a novel topology specifically for distributed matrix multiplication. We incorporate multicast and two tricks into our network design and demonstrate their effectiveness through several experiments. We also compare our topology with Mesh to highlight its suitability in the context of distributed matrix multiplication.

6 Division of Labor

Student A: topology, routing(50%), tricks, experiment design, presentation(50%), report(50%).
Student B: multicast, routing(50%), presentation(50%), report(50%).
In summary, we complete the entire project with equal contributions.